

ASSIGNMENT-3.1

2303A51928

BATCH-30

Task 1: Zero-Shot Prompting (Palindrome Number Program)

PROMPT : Generate a Python function that checks whether a given number is a palindrome

CODE and OUTPUT :

```
assignment3-1.py > ...
1  #Generate a Python function that checks whether a given number is a palindrome
2  def is_palindrome(n):
3      num_str = str(n)
4      reversed_str = num_str[::-1]
5      return num_str == reversed_str
6  # Test the function
7  print(is_palindrome(121)) # True
8  print(is_palindrome(-121)) # False
9  print(is_palindrome(10)) # False
10 print(is_palindrome(12321)) # True
11 |
```

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
True
False
False
True
PS C:\200126>
```

Justification : In zero-shot prompting, the model receives only the problem statement without any supporting examples. This checks the AI's basic understanding of the concept. While generating a palindrome-checking function, the AI typically uses simple logic such as reversing the number or converting it into a string. However, due to the lack of constraints or examples, the solution may ignore important edge cases like negative values, single-digit numbers, or invalid inputs. This indicates that although zero-shot prompting works well for simple tasks, it may not produce fully robust or real-world-ready code without additional guidance

Task 2: One-Shot Prompting (Factorial Calculation)

PROMPT : Generate a Python function to compute the factorial of a given number.

Example: Input: 5 → Output: 120

CODE and OUTPUT :

```
12 #Generate a Python function to compute the factorial of a given number. Example: Input: 5 → Output: 120
13 def factorial(n):
14     if n < 0: return "Factorial is not defined for negative numbers"
15     elif n == 0 or n == 1: return 1
16     else:
17         result = 1
18         for i in range(2, n + 1):
19             result *= i
20         return result
21
22 # Test the function
23 print(factorial(5)) # 120
24 print(factorial(0)) # 1
25 print(factorial(-3)) # "Factorial is not defined for negative numbers"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
120
1
Factorial is not defined for negative numbers
PS C:\200126>
```

Justification : One-shot prompting provides the AI with one example along with the task description. The given example (Input: 5 → Output: 120) helps the model clearly understand the expected result and logic. Compared to zero-shot prompting, this approach leads to more accurate implementations with correct looping or recursion. The presence of a single example reduces ambiguity and improves correctness, showing that even minimal guidance can significantly enhance the quality of AI-generated code.

Task 3: Few-Shot Prompting (Armstrong Number Check)

PROMPT : Generating a Python function to check whether a given number is an Armstrong number

CODE AND OUTPUT:

```

27 #Generating a Python function to check whether a given number is an Armstrong number
28 def is_armstrong(n):
29     num_str = str(n)
30     num_digits = len(num_str)
31     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
32     return armstrong_sum == n
33 # Test the function
34 print(is_armstrong(153)) # True
35 print(is_armstrong(9474)) # True
36 print(is_armstrong(123)) # False
37 print(is_armstrong(0)) # True
38

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

True
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
120
1
Factorial is not defined for negative numbers
PS C:\200126> ^C
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
True
True
False
True
PS C:\200126> 

```

Justification : Few-shot prompting includes multiple examples, which strongly influences how the AI interprets the problem. By seeing both Armstrong and non-Armstrong cases, the model better understands digit extraction, power calculations, and decision-making logic. This results in cleaner, more structured, and mathematically accurate code. Additionally, the AI performs better when handling different test cases. Overall, providing several examples improves reliability and reduces logical mistakes, making the solution more suitable for real-world use

Task 4: Context-Managed Prompting (Optimized Number Classification)

PROMPT : Generate an optimized Python program that classifies a number as prime, composite, or neither

CODE AND OUTPUT:

```

38 #Generate an optimized Python program that classifies a number as prime, composite, or neither
39 def classify_number(n):
40     if n <= 1:
41         return "Neither"
42     elif n <= 3:
43         return "Prime"
44     elif n % 2 == 0 or n % 3 == 0:
45         return "Composite"
46
47     i = 5
48     while i * i <= n:
49         if n % i == 0 or n % (i + 2) == 0:
50             return "Composite"
51         i += 6
52     return "Prime"
53

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

True
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
120
1
Factorial is not defined for negative numbers
PS C:\200126> ^C
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
True
True
False
True
PS C:\200126> ^C
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py
PS C:\200126> █

```

Justification : Context-managed prompting supplies detailed instructions, constraints, and performance expectations. By specifying input validation, efficiency requirements, and classification rules, the AI produces a well-optimized and complete program. The generated solution correctly handles special cases such as 0, 1, and negative numbers, and applies efficient logic by checking divisibility only up to $\sqrt{n}$. Compared to simpler prompting techniques, this method results in professional-quality, optimized, and reliable code, highlighting the importance of clear context for complex tasks

Task 5: Zero-Shot Prompting (Perfect Number Check)

PROMPT : Generate a Python function that checks whether a given number is a perfect number

CODE and OUTPUT :

```

53 # Generate a Python function that checks whether a given number is a perfect number
54 def is_perfect(n):
55     if n < 1:
56         return False
57     divisors_sum = sum(i for i in range(1, n) if n % i == 0)
58     return divisors_sum == n
59
60 # Test the function
61 print(is_perfect(6)) # True
62 print(is_perfect(28)) # True
63 print(is_perfect(12)) # False
64 print(is_perfect(0)) # False
65
66

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

True

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py

120

1

Factorial is not defined for negative numbers

PS C:\200126> ^C

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py

True

True

False

True

PS C:\200126> ^C

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment3-1.py

True

True

False

False

PS C:\200126>

Justification : In this zero-shot scenario, the AI depends entirely on its prior knowledge to generate the solution. The code usually identifies divisors and calculates their sum correctly. However, it may include inefficiencies such as checking all numbers up to n or failing to validate nonpositive inputs. This demonstrates that while zero-shot prompting can generate functional code, it often lacks optimization and thorough validation. Therefore, it is better suited for basic demonstrations rather than efficient or production-level solutions.